Cache-Aware & Cache-Oblivious Algorithms for String Sorting

Kausheya Roy Harshit Lalwani

AlgoEngineering Project

May 2026

Outline

Problem Statement

Goal

Sort N strings while **minimising I/O operations** $Q(N; M; B)$ between fast cache (M) and slower memory (block size B).

- **Success metric:** approach the scan bound $\mathrm{Scan}(N) = O(N/B)$
- Modern CPUs: L1 48 KiB, L2 1.25 MiB, L3 12 MiB $\Rightarrow$ memory hierarchy deeply affects runtime
- Standard sorts ignore cache $\Rightarrow$ $O(N \log N)$ cache misses on large N

Key insight

Engineering cache locality delivers more practical speedup than asymptotic optimality alone.

Memory Hierarchy & I/O Model

External-memory (EM) model

- Two levels: fast cache of size M, slow disk
- Data transferred in blocks of size B
- Cost = number of block transfers $Q(N; M; B)$

Cache-oblivious model

- Same two-level structure but algorithm *does not* know M or B
- Optimal for all levels simultaneously

Test machine (WSL2)

CPU	i5-12450H
L1d	48 KiB/core
L2	1.25 MiB/core
LLC	12 MiB
RAM	7.6 GiB
OS	kernel 6.6.87

Theoretical Complexity Summary

Algorithm	Time	I/O $Q(N; M; B)$	Model
`std::sort`	$O(N \log N \cdot \bar{\ell})$	$O\left(\frac{N}{B} \log N\right)$	agnostic
Multikey QS	$O(D + N \log N)$	$O\left(\frac{N}{B} \log N\right)$	agnostic
MSD Radix	$O(D)$	$O(D/B)$ if $\lvert\Sigma\rvert \leq M/B$	aware
Burstsort	$O(D + N \log L)$	$O\left(\frac{N}{B} \cdot \frac{D}{NL}\right)$	aware
C-Burstsort	$O(D + N \log L)$	$O\left(\frac{N}{B} \cdot \frac{D}{NL}\right)$	aware
Funnelsort	$O(N \log N)$	$O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$	oblivious

L: burst threshold; $\bar{\ell}$: mean string length.

$D = \sum d_i$: total distinguishing prefix length;

Cache-Aware: Burstsort & C-Burstsort

Burstsort idea:

- Trie with leaf *buckets* sized $L < M$
- In-bucket sort stays entirely in cache
- I/O dominated by trie traversal: near $O(N/B)$

C-Burstsort (arena variant):

- Single contiguous arena replaces heap allocation
- Prefix chars copied to dense key buffers
- Same O complexity, substantially lower constant

I/O bound

$$Q \approx O\left(\frac{N}{B} \cdot \frac{D}{N \cdot L}\right)$$

Choose $L < M$: bucket sorts free; trie scan $= O(N/B)$

Cache-Aware: MSD Radix Sort

- Count frequencies at depth d, distribute into $|\Sigma|$ buckets, recurse per bucket
- I/O: $O(D/B)$ when $|\Sigma| \leq M/B$ — each pass is a scan
- **Depth sensitivity:** mean recursion depth $= D/N$; high-LCP datasets (`wiki_heavy`) cause many passes
- Bucket thrashing when $|\Sigma| > M/B$: every write hits a different cache line

Fixed in implementation

Distribute writes at `lo + count[c]` (not absolute 0); copy-back uses the same $[lo, hi]$ window.

Cache-Oblivious: Lazy Funnelsort

- K-way merge sort, $K = \lceil N^{1/3} \rceil$
- Base case: 8 192 strings ($\approx 114\,\text{KB} \leq$ L2 per core)
- Proof sketch: recursion bottoms out when $n \cdot \bar{\ell} \leq M$; subsequent merges incur *zero* extra I/O
- Holds for *any* M and B

Optimal I/O

$$Q = O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$$

Limitation

Comparison-based $\Rightarrow$ wall-clock $\approx$ `std::sort`; advantage is provable I/O bound without knowing M, B.

Four Datasets

Synthetic

- **random** — length 4–24, uniform chars; minimal LCP depth; tests $N \log N$ base case
- **prefix_heavy** — shared long common prefix; stresses D; exposes MSD Radix thrashing

Wikipedia titles (up to 5×10^6)

- **wiki_random** — uniform sample; moderate LCP; representative real-world vocabulary
- **wiki_heavy** — "List of..." filtered; maximises D; worst case for depth-sensitive algorithms

Why four datasets?

Each dimension isolates a different complexity parameter: N, D/N, $|\Sigma|$, and real-world prefix distribution.

Experimental Setup

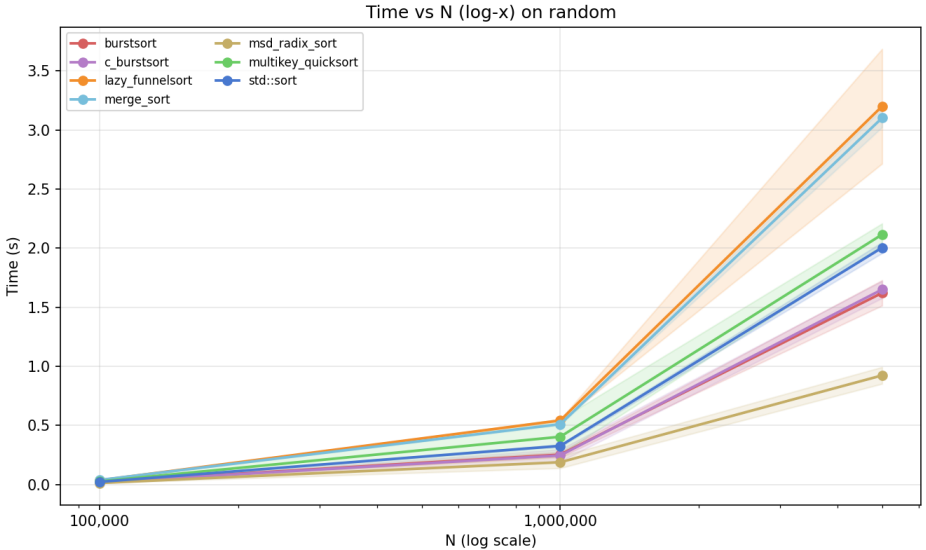
Methodology

- 10 seeds per (algorithm, dataset, N)
- 2 warm-up runs discarded (cold-cache bias)
- Timer: `std::chrono::steady_clock` around sort only
- Correctness: `std::is_sorted` after every run
- Cache counters: `perf stat` at $N \in \{10^4, 10^5, 10^6\}$
- CPU pinned: `taskset -c 0`, frequency scaling off

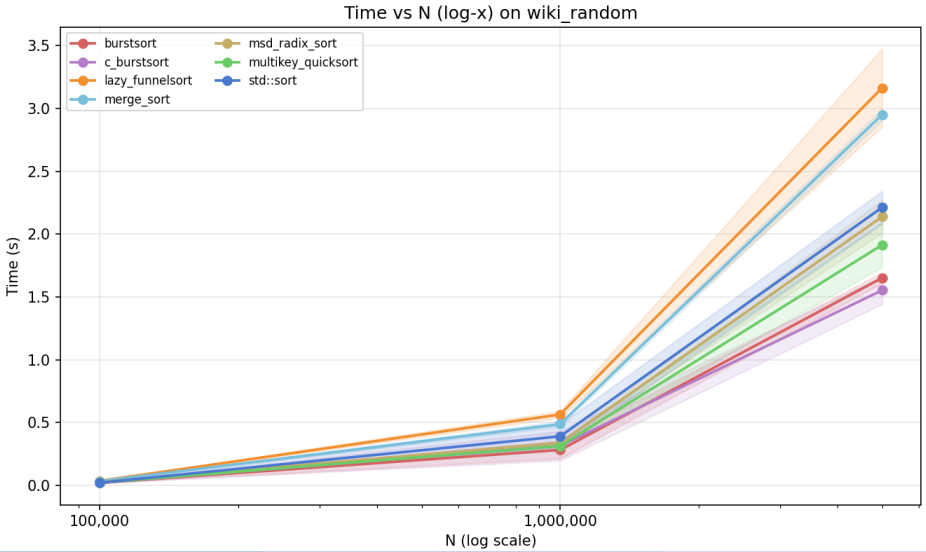
Hardware (WSL2)

CPU	i5-12450H
L1d	48 KiB/core
L2	1.25 MiB/core
LLC	12 MiB
RAM	7.6 GiB
Kernel	6.6.87-WSL2
Flags	`-O3 -march=native`

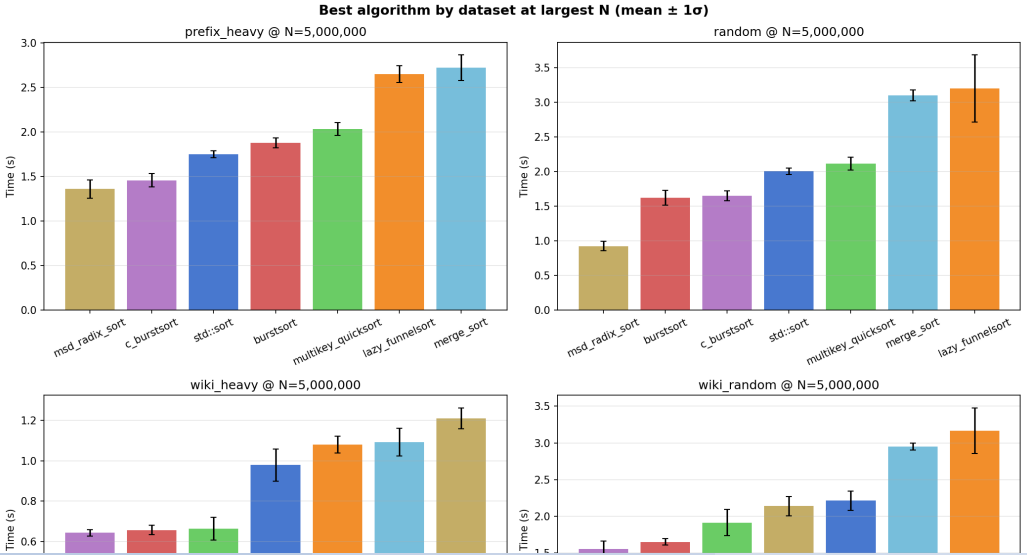
Wall-clock Time vs. N — `random` Dataset



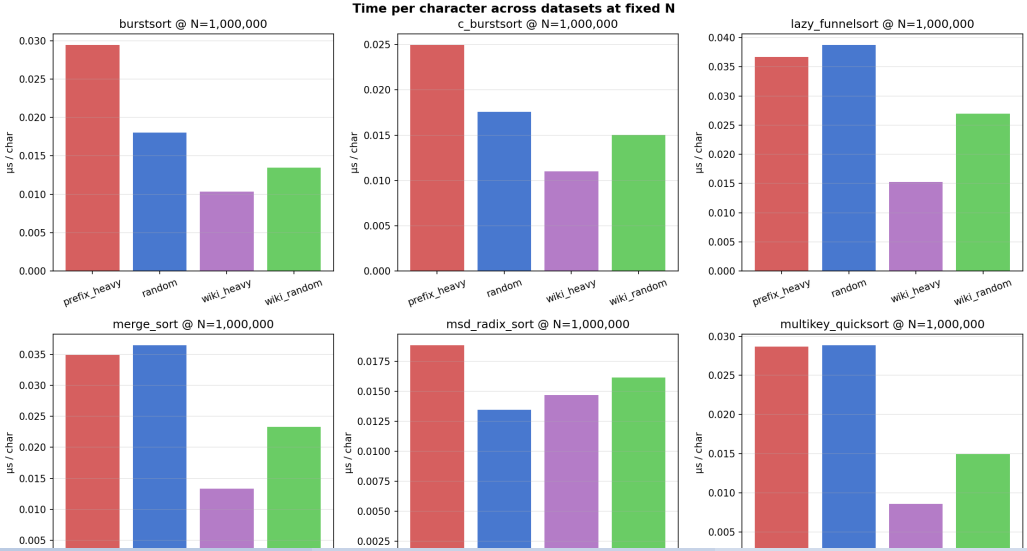
Wall-clock Time vs. N — `wiki_random` Dataset



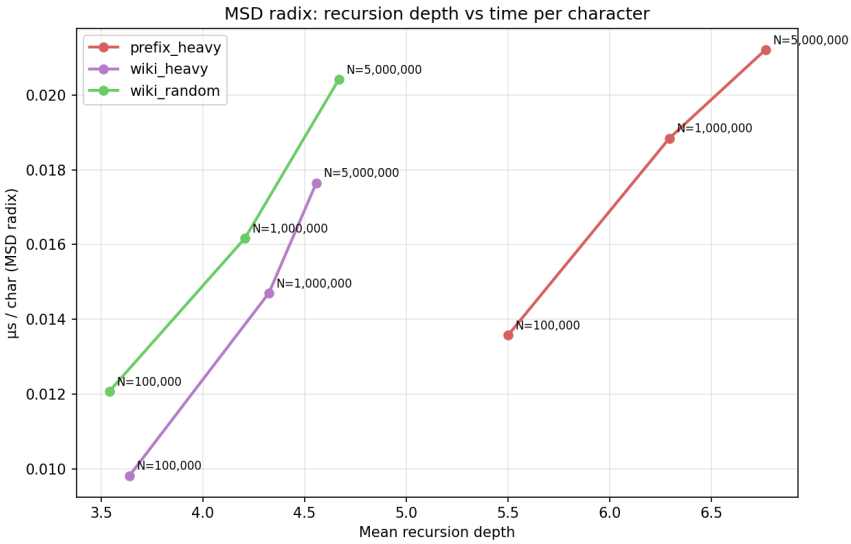
Best Algorithm at Largest N (All Datasets)



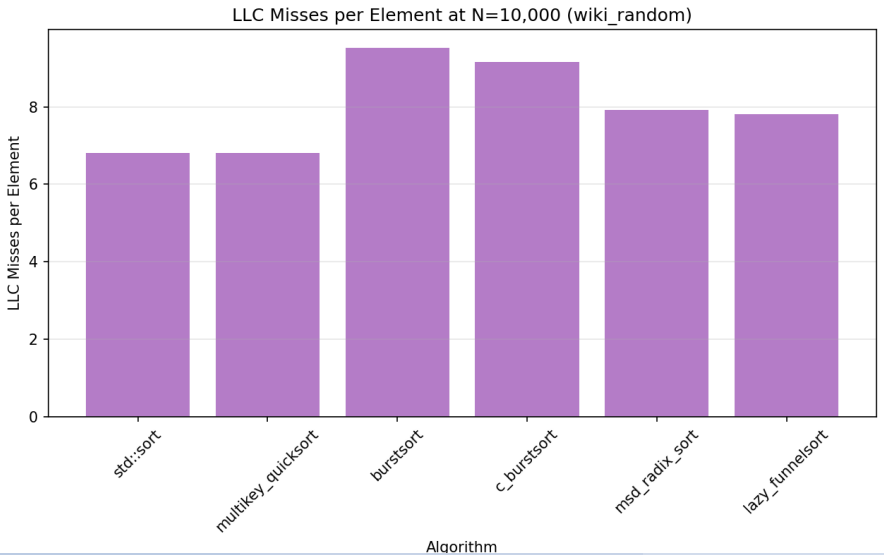
Time per Character at $N = 10^6$



MSD Radix: Recursion Depth vs. Time per Character



Cache Profiling: LLC Misses per Element



Key Observations

1. **Cache-aware $\succ$ cache-agnostic at large N.** Burstsort/C-Burstsort near-linear; `std::sort` super-linear beyond LLC (12 MiB here).
2. **Arena allocation matters.** C-Burstsort beats Burstsort by a consistent constant factor (dense key buffers = better spatial locality).
3. **MSD Radix is depth-limited.** Recursion depth $\propto D/N$; directly causes outlier cost on `wiki_heavy` and `prefix_heavy`.
4. **Funnelsort $\approx$ `std::sort` in wall-clock.** Comparison-based design; value is provably optimal I/O without knowing M or B.
5. **Architectural lesson.** Bucket-size control ($L < M$) + contiguous layout (arena) $>$ theoretical optimality alone.

Thank You

Questions?

Report	`main.tex` / `main.pdf`
Slides	`string_sorting_beamer.tex`
Code	`cpp/src/algorithms/`
Analysis	`Analysis/cache_aware_analysis.py`

Kausheya Roy **Harshit Lalwani**
AlgoEngineering · May 2026